SPETC v10 — Maintainer's Guide

Mauro Barbieri
mauro.barbieri@pm.me

2007–2026

Abstract

Internal documentation for whoever maintains or extends SPETC: the architecture and data flow, a module-by-module reference of every source file and its public functions, the on-disk data and configuration formats, the test suites, recipes for the most common modifications, and how to run and package the program on Windows and macOS.

Contents

1 Architecture

1.1 Who this guide is for

This guide assumes you have never opened the SPETC source. It explains what the program is made of, how a calculation flows through it, where every kind of file lives, and then — once the map is in your head — how to change things safely: fix a bug, add a physical effect, add data, publish a release. Read this section and Sect. 2 before touching anything; the recipes of Sect. 4 then tell you exactly which file to open for each kind of change.

1.2 The shape of the program

SPETC is deliberately *not* a framework: it is sixteen flat Python modules in one directory, no packages, no plugins, no configuration language. Two classes of module exist, distinguished by one strict rule:

> **Physics modules never import the GUI, and only `etc_gui.py` imports Tkinter.**

The consequence is that everything scientific is importable, testable and scriptable from a plain Python session or the test suite, with no window involved; the GUI is a thin, replaceable skin. The batch tool `spetc_batch.py` is the proof: it produces complete results using the same engines and zero Tk.

Within the physics side, the modules form five layers, each depending only on the layers below it:

1. **Foundations.** `spectral_utils.py` defines the one data structure the whole program shares: a `SpectralCurve`, a validated pair of (strictly increasing wavelength in Å, value) arrays, with three interpolation policies that encode the project's coverage philosophy — `interpolate_checked` (raise outside coverage: for calibration), `interpolate_zero_filled` (assume zero outside: for detector-side quantities like QE), `interpolate_edge_extended` (extend the edge value: for the atmosphere, which does not become opaque where a file ends). It also reads two-column text files and atmospheric FITS tables. `observing_conditions.py` is a set of pure functions for the atmosphere-as-noise: scintillation (Young's law, in both per-exposure and per-second-of-variance form), Filippenko differential refraction, and ADC quantization noise. Neither imports anything else of the project except each other.

2. **Physics core.** `etc_physics.py` is the radiometric heart: magnitude systems and zero points, synthetic photometry in the SVO convention, the template flux calibration (the m_{v0} machinery), the electron-rate integral, the PSF models (Gaussian and Moffat, with their aperture and slit couplings) and the CCD signal-to-noise equation. `sky_brightness.py` and `sky_background.py`

together model the sky: the first owns the component physics (van Rhijn airglow, zodiacal light, starlight, the Krisciunas–Schaefer Moon) and the shared constant tables; the second owns the *normalization policy* — which component sets the level at a given Sun altitude (dark table, SQM reading, Weaver daylight, twilight blend). `ephemeris.py` wraps Astropy for everything positional: refraction-corrected altitude–azimuth tracks, Pickering airmass, Sun/Moon positions and phase, parallactic angle, and coordinate/time parsing.

3. **Engines.** `photometry.py` and `spectroscopy.py` each expose one class with one compute method. They accept what we call the *four-dictionary context* — `telescope` (a plain dict of numbers and optional curves), `detector` (a `Detector` instance from `detector.py`), `atmosphere` (airmass, seeing, PSF model, elevation, transmission curve), `sky_model` (brightness, calibration, geometry) — plus per-call arguments, and return plain results: a dict for photometry, a `pandas.DataFrame` with metadata in `.attrs` for spectroscopy. This context is the API contract of the whole program: anything that can build those four objects can run the ETC. `solvers.py` inverts the S/N equation in closed form and plans frame stacks.

4. **Catalogues.** `star_catalog.py` parses the template catalogue and loads spectra (ASCII and CALSPEC-style FITS); `filter_catalog.py` parses SVO VOTable filter profiles and lists the available filters; `config_manager.py` loads the built-in observatory presets.

5. **Presentation and tools.** `etc_gui.py` (the Tk application) and `matplotlib_plots.py` (the native plot windows) on the interactive side; `spetc_batch.py`, `make_filter_profile.py` and `add_template.py` as command-line tools; `self_check.py` and `tests/test_spectral_pipelin` as the test entry points.

1.3 One calculation, end to end

To understand the code, follow one press of Run ETC through it.

1. **Input validation.** `ETCGUI._run_etc` reads every relevant Tk variable, converts and range-checks it, and constructs the target coordinates. Anything invalid raises `ValueError` with a human sentence; the GUI catches it and shows it — this is the program's error policy everywhere: *raise early, with the offending number in the message, and never substitute a silent default*.

2. **Ephemerides.** `ephemeris.compute_target_track` produces, for a window of ± 18 hours around the chosen time, the refraction-corrected altitude, azimuth, Pickering airmass, parallactic angle, and the Sun and Moon positions, separations and phase at five-minute steps. Everything time-dependent downstream is evaluated on this track.

3. **Sky models.** `_sky_models_for_track` converts the target's ICRS coordinates to ecliptic and galactic latitude once, then builds one sky-model dictionary per time sample: the observed surface brightness in the observing band (from the ING physical model, the user's SQM reading, or a fixed value, plus the Moon term), the photometric zero point it is expressed in, and the spectral colour model used by spectroscopy.

4. **Engine loop.** For every visible time sample, the GUI builds the four-dictionary context and calls the engine. In target-S/N mode it first calls the engine at 1 s to obtain rates, inverts the S/N equation in closed form (`solvers.exposure_time_for_snr`, scintillation included as an extra variance rate), and calls the engine again at the solved exposure. Full spectra are computed only at the selected time and at the plot-slider samples; all other samples evaluate just the reference-wavelength bin, which keeps a night's planning at $R = 100\,000$ responsive.

5. **Aggregation and display.** The per-time values become the time-series DataFrame (one row per sample: time, geometry, parallactic angle, airmass, S/N or required exposure, sky brightness,

extinction, saturation); the selected-time result becomes the results table; the stack planner and the differential-precision helper run on the selected-time rates; plots and the parameters/outputs log are refreshed; the configuration is saved.

1.4 Inputs and outputs at a glance

File	Direction	Role
`data/interpola.db.csv`	in	template catalogue (GUI list)
`data/bpgs/*`, `data/imported/*`	in	template spectra (ASCII/FITS)
`data/filters.list`, `data/filters/*.xml`	in	filter profiles (SVO VOTable)
`data/qe.dat`, curve files	in	QE, optics response, slit-R, gain table
`data/earth_atmospheric_transmission.fits`	in	measured zenith transmission
`etc_sites.json`, `observatories.json`	in/out	site lists
`etc_user_config.json`	in/out	full GUI state, rewritten each run
instrument profile JSON + curve copies	in/out	portable per-instrument setup
result CSV / time-series CSV	out	full numerical results (every engine field)
batch JSON	in	headless run description
batch result CSV + HTML summary	out	headless outputs

The complete field-by-field specification of every input format is in the *User Guide*, Sect. 6, written to be sufficient for producing the files from scratch; this guide does not duplicate it, but Sect. 3 adds the parsing internals a maintainer needs.

1.5 Project conventions

Wavelengths are Ångström internally, everywhere; every user-facing curve declares its unit explicitly and is converted once at load. All radiometric integrals carry `astropy.units` and are reduced to floats only at the engine boundary. Constants are named after their paper (`SOLAR_MINIMA`, `MOON_FUDGE`); every empirical relation cites its reference in the docstring; every physical claim in the code has a regression test pinned to a published number, not to the code's own output. Azimuth is N$=0°$/E$=90°$; times are UTC internally and converted only for display.

2 Module reference

2.1 `spectral_utils.py`

Unit-aware spectral curves. `SpectralCurve` (frozen dataclass, strictly increasing Å grid); `make_curve`/`as_curve` validate and convert; `require_coverage` and `interpolate_checked` enforce the no-extrapolation rule; `interpolate_zero_filled` exists only for diagnostic plots; `load_two_column_curve` reads whitespace/comma text files; `load_fits_transmission_curve` reads atmospheric FITS tables (direct transmission columns, Paranal `EXTINCTION` mag/airmass converted once, or a $2 \times N$ image with `CUNIT1`).

2.2 `observing_conditions.py`

Pure functions, no state: `scintillation_fractional_rms` / `scintillation_variance_e2` (Young 1967 law), `scintillation_variance_rate_e2_s` (the same variance expressed as a time-linear rate, $(\dot{S}C)^2/2$, so closed-form solvers can absorb it as an extra Poisson-like term), `digitization_noise_e` ($g/\sqrt{12}$), `refractive_index_minus_one` and `differential_refraction_arcsec` (Filippenko 1982).

2.3 etc_physics.py

magnitude_f_lambda (zero-point F_λ), synthetic_magnitude (SVO photon/energy convention), calibrated_template_magnitude (CALSPEC m_{v0} + synthetic colour scaling), collecting_area, atmospheric_transmission (T^X), instrument_transmission, electron_rate (Eq. 4 of the physics document), psf_encircled_energy and psf_slit_throughput (Gaussian/Moffat; the Moffat slit coupling is the Student-t CDF with $\nu = 2\beta - 2$), and snr (CCD equation with an extra_variance_e2 hook). The Gaussian-only wrappers gaussian_encircled_energy and slit_throughput are kept for backward compatibility.

2.4 sky_brightness.py / sky_background.py

sky_brightness.py owns the shared constants (solar minima, U–K tables, lunar factors) and the component physics: van_rhijn_factor, zodiacal_s10, starlight_s10, dark_sky_position_correctio moonlight_scattering_function and the nine-band sky_brightness_total. sky_background.py supplies the observed normalization sky_magnitude_vega: dark table + position/van-Rhijn correction below $-18°$ Sun altitude, Weaver daylight above the horizon, linear flux blend in twilight. The GUI stitches them: the ING normalization sets the level, the Benn–Ellison/K&S bands set the colour and the Moon term.

2.5 ephemeris.py

compute_target_track is the workhorse: refraction-corrected AltAz frames (site_pressure_hpa/site_te ISA at elevation), pickering_airmass above the $5°$ cutoff, Sun/Moon positions, Moon phase and separation, and parallactic_angle_deg per sample. Parsers and converters (parse_ra, sexagesimal formatters, frame converters) and the find_* rise/set helpers complete the module.

2.6 photometry.py / spectroscopy.py

Each exposes one class with one compute method taking the four-dictionary context plus per-call arguments. PhotometryETC.compute_photometry_single handles point and extended geometry, the full noise budget and saturation; SpectroscopyETC.compute_spectroscopy handles slit (PSF + dispersion-offset losses) and slitless (grating physics via module function grating_dispersion_aa_per_pixel) modes and returns a DataFrame whose attrs carry per-run metadata (mode, R, sky brightness, noise terms, dispersion).

2.7 detector.py, solvers.py

Detector stores pixel size, gain, full well, bit depth, read noise, dark current; counts_to_adu and saturation_flag (FULL_WELL/ADC/BOTH/NONE); load_gain_table reads the per-setting CMOS tables (Sect. 3); sensor_type/osc_channel select mono or a Bayer channel, whose channel_fill_fracti (G 1/2, R/B 1/4) both engines apply to aperture rates and channel-pixel counts but not to the peak pixel. solvers.exposure_time_for_snr inverts the CCD equation in closed form, with extra_variance_rate_e absorbing time-linear non-Poisson terms (scintillation); solvers.plan_stack produces the sub-exposure/frame-count plan with the read-noise penalty and the sky-limited frame length, on the same closed form.

2.8 make_filter_profile.py, spetc_batch.py

Standalone tools sharing the physics modules. make_filter_profile turns a two-column transmission file into a full SVO-format VOTable pair: measure_band_quantities evaluates every SVO characterising quantity (Min/Max/Peak on the tabulated grid, per SVO's literal definitions) and synthetic_vega_zero_point_jy computes the Vega zero point from the bundled CALSPEC

spectrum; validated against the shipped 2MASS.H profile in the test suite. `spetc_batch.run_batch` drives both engines headless from a JSON description and renders the result CSV plus a self-contained HTML summary with an embedded S/N figure — it is also the reference example of the four-dictionary engine API.

2.9 `star_catalog.py`, `filter_catalog.py`, `config_manager.py`

Catalogue parsing (`interpola.db.csv` → `StarRecord`; spectrum loading with monotonicity cleanup; `load_fits_spectrum` reads CALSPEC/STScI binary tables with TUNIT-aware wavelength conversion, dispatched by file extension), SVO VOTable parsing (`FilterProfile` with zero point, `DetectorType`, metadata in Å; `BLANK` special-cased to the AB scale), and the built-in observatory presets from `observatories.json`.

2.10 `etc_gui.py`, `matplotlib_plots.py`

The Tk application: `_var(key, default)` registers every setting for automatic JSON persistence; `_run_etc` orchestrates; the helper methods `_atmosphere_dict`, `_source_geometry_kwargs`, `_sky_models_for_track`, `_photometry_time_series`, `_spectroscopy_time_series` and `_build_time_series_dataframe` are the seams to modify first. `matplotlib_plots.ETCPlotWindow` renders the metric/altitude/sky-path (and spectrum) panels with the manual time slider.

3 On-disk formats

3.1 `data/` directory

`interpola.db.csv` Header line then comma-separated rows: `nrow, Vmag, B-V, name, spt, filename`; `filename` is relative to `data/` (subdirectories allowed, e.g. `bpgs/bpgs_92.ascii`). `Vmag` is the CALSPEC visual magnitude represented by the file (m_{v0}).

template spectra Two-column ASCII (wavelength [Å], F_λ) or CALSPEC/STScI-style FITS binary tables (`WAVELENGTH`/`FLUX` and common variants; `TUNIT` in Angstrom, nm or micron; extra columns ignored), dispatched by extension. Non-finite and non-positive rows are dropped; wavelengths are sorted and deduplicated on load. `add_template.py` imports files or directories, computing the synthetic $m_{v0}/B-V$ and appending the catalogue rows.

`qe.dat` Two-column wavelength/QE; the wavelength unit is declared in the configuration (`Angstrom`/`nm`/`um`).

`filters.list` One entry per line naming SVO profiles; entries with or without the `filters/` prefix and `.xml` suffix all resolve. Each logical filter needs `<name>_Vega` and `<name>_AB` VOTables carrying `MagSys`, `ZeroPoint` [Jy], `DetectorType`, `WavelengthUnit` and the transmission table. `BLANK.xml` (unity 3000–26000 Å, AB) is optional and auto-detected.

`earth_atmospheric_transmission.fits` Optional; accepted layouts in Sect. 2 (`spectral_utils`).

CMOS gain tables One row per gain setting, whitespace- or comma-separated, # comments allowed: `setting e-/ADU read_noise_e full_well_e`. Loaded through `detector.load_gain_table`; the GUI persists the path in `gain_table_path` and restores it at start-up.

Batch run descriptions JSON, see `examples/`: top-level `mode`, `telescope`, `detector`, `atmosphere`, `sky` (`fixed_ab`/`sqm`), `target`, `photometry` or `spectroscopy` blocks, optional `target_snr` and `stack_sub_exposure_s`.

3.2 Configuration and profiles

`etc_user_config.json` is a flat string-to-string map of every registered GUI variable plus `schema_version`; it is rewritten after each successful run and on exit. Instrument profiles are the same keys restricted to the instrument subset (`_instrument_profile_keys`), saved with relative copies of the QE, response and slit-R curves beside them. `etc_sites.json` holds user sites (`name/lat/lon/elev/utc_offset_h/timezone`) `observatories.json` the built-in presets. When adding a new persistent setting, register it with `self._var("key", default)` — persistence is automatic; bump `schema_version` only for incompatible changes.

4 Modification recipes

Each recipe names every file to touch, in order, and ends with the test obligation. The general pattern is always the same: implement the physics as a pure function in the right layer, thread it through the engine(s), expose it in the GUI with a persisted variable, and pin it with a test.

4.1 Add a filter

Files touched: `data/filters.list`, `data/filters/`. For a filter that exists in the SVO Filter Profile Service, download the VOTable of *both* calibrations (the Vega page and the AB page of the same filter), save them as `<System.Band>_Vega.xml` and `<System.Band>_AB.xml` in `data/filters/`, and append the two names to `data/filters.list`. No code changes: the selector is rebuilt from the list at start-up and every number the program needs (zero point, detector type, pivot, transmission) is inside the files. For a filter SVO does not carry, produce the pair from a transmission curve with `make_filter_profile.py` — it prints the exact lines to append. If the program refuses a profile, the parser's error message names the missing `PARAM`; the four required ones are `MagSys`, `ZeroPoint` (+`ZeroPointUnit Jy`), `DetectorType` and `WavelengthUnit`.

4.2 Add template spectra

Files touched: `data/`, `data/interpola.db.csv` — both via the tool. Run `python3 add_template.py <file-or-directory>`. The tool computes the one non-obvious catalogue field, m_{v0} (the synthetic Vega V magnitude of the file — get this wrong and every prediction from that template is off by the same amount), writes the row, and copies the file. Do it by hand only when you understand the m_{v0} convention (User Guide Sect. 6.1). To *remove* a template, delete its catalogue line; the spectrum file may stay.

4.3 Add an observing site

Use Save current site in the GUI, which writes `etc_sites.json`; or edit that JSON directly (schema in the User Guide Sect. 6.7). `observatories.json` is the shipped default set: add there only sites that should reach every user of a release.

4.4 Add a noise term

Files touched: `observing_conditions.py`, `photometry.py`, `spectroscopy.py`, possibly `solvers.py`, `etc_gui.py`, `tests/`. Write the term as a pure function returning a *variance in e^{-2}* given rates and conditions, with the reference in the docstring. In `photometry.compute_photometry_single` and in the S/N line of `spectroscopy.compute_spectroscopy`, add it to the `extra_variance` sums (grep for `scintillation_var` to find both spots). Ask the one question that decides solver integration: is the variance linear in exposure time? If yes (like scintillation), also add its rate to the `extra_variance_rate_e2_s` arguments of `exposure_time_for_snr` and `plan_stack` at the call sites in `etc_gui.py` and `spetc_batch.py`, or target-S/N mode will silently ignore it. If it needs a new user input, register the Tk variable with `self._var("key", default)` (persistence

is then automatic) and pass it through `_atmosphere_dict`. Finish with a regression test pinned to a published magnitude of the effect and a line in the physics document's noise table.

4.5 Add a PSF model

Files touched: `etc_physics.py`, `etc_gui.py`, `tests/`. Add a branch to `psf_encircled_energy` (circular integral) and `psf_slit_throughput` (the 1-D marginal CDF, analytic if you can find it — the Moffat case shows the pattern with the Student-t — numeric otherwise; keep the `offset_arcsec` argument vectorised, the dispersion-loss code calls it with arrays). Add the model name to the GUI combobox. Both engines pick it up automatically because they read `psf_model` from the atmosphere dictionary. Test it against the Gaussian in a limiting case.

4.6 Change or extend the sky model

Component physics (a better zodiacal model, a site-specific airglow coefficient) belongs in `sky_brightness.py`; *when* each component sets the level belongs in `sky_background.py`; how the per-time dictionaries are assembled for the engines is `_sky_models_for_track` in the GUI. A future fully spectral sky model has a ready-made slot: put a curve under the `spectral_sky_f_lambda` key of the sky-model dictionary and the spectroscopy engine uses it as-is, bypassing the broad-band colour machinery.

4.7 Add a GUI control

Register the variable (`self._var`), place the widget in the right column builder (`_build_*_column`), consume the value in `_run_etc` or a helper, and — if the physics sees it — extend the run-parameters log in `_show_info` so the assumption is visible in the output record. Never put computation in the GUI: compute in an engine or module function and call it.

4.8 Add an output quantity

Compute it in the engine and put it in the result dict / DataFrame — it then reaches the result CSV automatically. To show it in the results table, add a `(key, heading)` pair to `ETCGUI._RESULT_COLUMNS`; to show it per time sample, add a column in `_build_time_series_dataframe`; to show it in the log, add a line to the `_append_info_outputs` call. Document it in the README output table and the User Guide.

5 Testing

Two entry points, both dependency-light and network-free:

```
python3 self_check.py
python3 tests/test_spectral_pipeline.py
```

`self_check.py` is a synthetic end-to-end smoke test of both engines on the 358-mm reference configuration; it pins the scintillation-limited photometric S/N ($\sim 1.3 \times 10^3$) and the per-element spectroscopic S/N (hundreds). `tests/test_spectral_pipeline.py` holds the unit-level regressions: unit conversion of user curves, SVO photon/energy semantics, hard coverage failures, FITS atmosphere layouts, and the v10 physics checks — Krisciunas–Schaefer scattering values, ecliptic symmetry and position dependence of the sky, van Rhijn bounds, Pickering vs. sec z, parallactic-angle sign convention, Moffat wings and decentred coupling, Filippenko dispersion scale, scintillation and quantization magnitudes, SA100/SA200 dispersion, and extended-source area fractions. The file inserts the repository root on `sys.path`, so it runs from any working directory; it is also `pytest`-compatible (every test is a plain `test_*` function).

Rules: any bug fix gains a regression test that fails on the pre-fix code; any new physical relation gains a test pinned to a published number, not to the code's own output. The GUI is deliberately untested — keep logic out of it and in the engines, where it is testable headlessly.

6 Working with Git, GitHub and Codeberg

6.1 Daily workflow

The repository is a standard Git project; the default branch is `main` and it should always pass both test programs. The safe cycle for any change, however small:

```
git clone https://github.com/<owner>/SPETC.git
cd SPETC
git switch -c fix/moon-model        # a branch per change
# ... edit ...
python3 self_check.py                # both must pass before every commit
python3 tests/test_spectral_pipeline.py
git add -A
git commit -m "Fix Moon scattering normalization

One paragraph on what changed physically and why; cite the
reference if a formula changed."
git push -u origin fix/moon-model
```

Then open a pull request on the web interface, from your branch into `main`, and merge it after review (even self-review: reading your own diff in the PR view catches a remarkable number of slips). Branch names with a prefix (`fix/`, `feature/`, `docs/`) keep the branch list legible. Never commit directly to `main`, and never commit a red test suite: the suite is the contract that the physics still holds.

Commit messages follow the repository's existing style: a one-line imperative summary, a blank line, then prose paragraphs explaining the *physics or design* of the change (not a file list — the diff shows the files). When a commit fixes a formula, the message names the paper.

6.2 Keeping a fork or a second remote (Codeberg)

To host the project on Codeberg (or any second forge) as a mirror or as the primary, add it as a second remote — Git has no notion of "the" server:

```
git remote add codeberg https://codeberg.org/<owner>/SPETC.git
git push codeberg main                    # first publication
git push codeberg --all --follow-tags     # everything, later
```

To push every branch to both forges routinely, either push twice (`git push origin && git push codeberg`) or configure one logical remote with two URLs:

```
git remote set-url --add --push origin \
    https://github.com/<owner>/SPETC.git
git remote set-url --add --push origin \
    https://codeberg.org/<owner>/SPETC.git
```

after which a single `git push` updates both. Authentication on both forges today means a personal access token used as the password over HTTPS, or an SSH key (`git@github.com:...`, `git@codeberg.org:...`); tokens are created in the respective web settings under *Developer settings / Applications*.

6.3 Releases

A release is a tag plus the artefacts:

```
git tag -a v10.1 -m "SPETC v10.1"
git push origin v10.1 && git push codeberg v10.1
```

Attach to the forge release page: the source archive (generated by the forge from the tag), the three compiled documentation PDFs, and the single-file executables of Sect. 7 for each platform you can build on. The release checklist: both test suites green; `README.md` version references updated; documentation recompiled; `etc_user_config.json` *not* committed with your personal values (it is runtime state — if it shows in `git status`, restore it).

6.4 What belongs in the repository

Source, tests, documentation sources and their compiled PDFs, the shipped `data/` directory, and the example/batch files. What does not: LaTeX build intermediates (already in `.gitignore`), `__pycache__`, personal configuration, large private spectra collections (users import those locally with `add_template.py`), and one-off analysis outputs. When in doubt: if a fresh clone would not need it to run or to rebuild the docs, leave it out.

7 Packaging: single-file executables for Linux, Windows and macOS

7.1 Principles

The code is pure Python with no compiled components of its own, so "porting" is a packaging exercise, done with PyInstaller. Three facts shape everything below. First, **PyInstaller does not cross-compile**: a Windows `.exe` must be built on Windows, a macOS app on macOS, a Linux binary on Linux — one build per platform, all from the same source and the same spec. Second, in `--onefile` mode the executable unpacks itself to a temporary directory at every start; bundled data is found under `sys._MEIPASS`, *not* next to the executable, so path handling needs the small shim below. Third, the program *writes* beside itself (`etc_user_config.json`, `etc_sites.json`, imported templates) — those files must live in a real, persistent, user-writable directory, never inside the bundle.

The clean policy, and the one assumed here: **bundle the read-only `data/` directory inside the executable, keep the writable files beside the executable**. Add this resolver near the top of `etc_gui.py` (and use it wherever the default data directory is composed):

```
import sys
from pathlib import Path

def bundled_data_dir():
    """data/ inside a PyInstaller bundle, or beside the source."""
    if getattr(sys, "frozen", False):        # running from PyInstaller
        return Path(sys._MEIPASS) / "data"
    return Path(__file__).resolve().parent / "data"
```

The GUI's data-directory field keeps working as the override for users with their own data trees.

7.2 Common build steps

On each platform, in a fresh virtual environment:

```
python3 -m venv build-env
. build-env/bin/activate        # Windows: build-env\Scripts\activate
pip install -r requirements.txt pyinstaller
pyinstaller --onefile --windowed --name spetc \
    --add-data "data:data" \
    --add-data "etc_sites.json:." \
    --add-data "observatories.json:." \
    etc_gui.py
```

(On Windows the `--add-data` separator is a semicolon: `"data;data"`.) The result lands in `dist/`. PyInstaller's built-in hooks handle NumPy, SciPy, Matplotlib and Astropy; two points need attention. Astropy must not try to download IERS tables from a frozen app — the code already sets `iers.conf.auto_download = False`, so the bundled table is used; verify by running the frozen app with the network off. And Matplotlib should be pinned to the TkAgg backend in the frozen app (`MPLBACKEND=TkAgg` via the spec's `runtime_hooks` or an environment default) to avoid backend probing at start-up.

Always test the frozen build with the checklist: starts offline; template list populated (bundled data found); a photometry and a spectroscopy run complete; Save instrument profile and site saving write real files next to the executable; the two CSV exports open.

7.3 Linux

The onefile binary from the recipe above runs on any distribution with glibc at least as new as the build machine's — so **build on the oldest distribution you want to support** (a container of Debian oldstable or Ubuntu LTS is the usual choice). Tk is bundled by PyInstaller. Users make it executable (`chmod +x spetc`) and run it. For desktop integration ship a `.desktop` file, or go one step further and wrap `dist/spetc` into an AppImage with `appimagetool`, which adds the icon/menu integration without changing anything in the build.

7.4 Windows

Build with the python.org interpreter (not the Microsoft Store one, whose sandbox breaks file dialogs and per-user paths). The recipe yields `dist\spetc.exe`; `--windowed` suppresses the console window. Two Windows realities to plan for: SmartScreen will warn on an unsigned executable — users click "More info → Run anyway", or you sign with an Authenticode certificate; and some antivirus products false-positive on PyInstaller onefile stubs — if reports come in, prefer `--onedir` (a folder instead of a single file, same command without `--onefile`, zipped for distribution), which is far less often flagged. High-DPI rendering is handled by Tk on Python $\geq$ 3.12; on older versions call `ctypes.windll.shcore.SetProcessDpiAwareness(1)` at start-up if the UI looks blurry.

7.5 macOS

Build on the oldest macOS you want to support, with the python.org universal2 interpreter (bundles its own Tcl/Tk; never use the system Python). With `--windowed` PyInstaller produces both a UNIX binary and a `spetc.app` bundle — distribute the `.app`, zipped. Gatekeeper will quarantine an unsigned download: users right-click → *Open* once, or remove the attribute (`xattr -dr com.apple.quarantine spetc.app`); for friction-free distribution you need an Apple Developer ID, `codesign --deep` and notarization with `xcrun notarytool` — mechanical but paid. An app built on Apple Silicon runs on Intel Macs only through Rosetta if built `universal2`; check `lipo -archs dist/spetc.app/Contents/MacOS/` if you support both.

7.6 Reproducibility

Commit the generated `spetc.spec` once it works — it *is* the build configuration — and record in the release notes the Python and PyInstaller versions of each artefact. A small `build.sh`/`build.bat` that creates the venv, installs pinned versions and runs PyInstaller turns the release build into one command per platform.